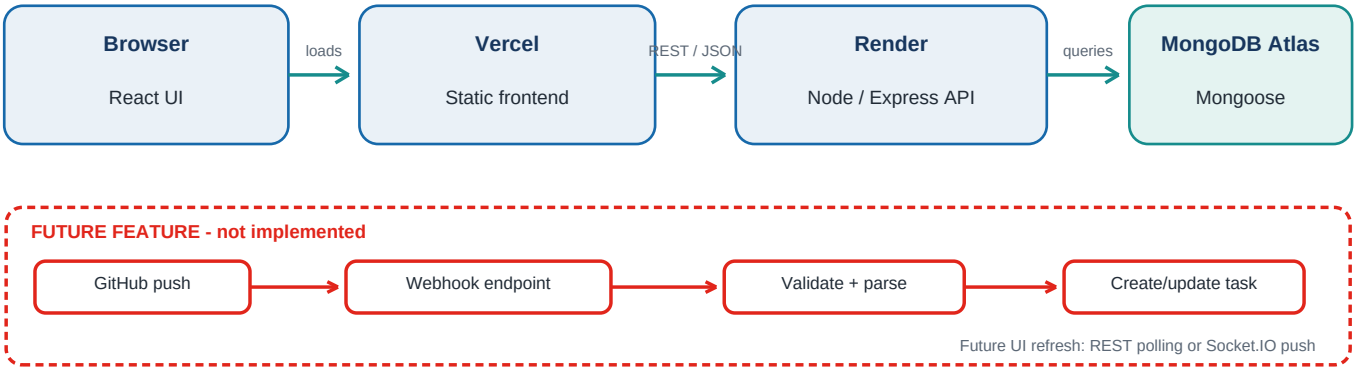
SprintStack Project Report

Implementation guide, assumed Vercel and Render deployment, and future GitHub automation

Current project	Deployment model in this report
React + Express REST API + MongoDB/Mongoose	**Vercel frontend + Render API + MongoDB Atlas**

SprintStack is a web-based task board for creating, organizing, and tracking work. The browser UI is a React single-page app. It sends JSON REST requests to an Express API, which authenticates users and stores tasks in MongoDB through Mongoose. This report describes the current code and a Vercel + Render deployment assumption; the GitHub commit-to-task integration is proposed future scope, not an existing feature.

SYSTEM ARCHITECTURE



Key distinction: the older Internship_assignment copy uses Socket.IO for task updates. SprintStack uses REST and refetches its task list after a write. It does not currently push another user's change into an already open board.

Report prepared 3 November 2026. No private database connection string, password, or signing key is included.

1. Project purpose and technology stack

The board helps a signed-in user organize tasks into To Do, In Progress, and Done columns. It combines task metadata, attachments, search and filters, and a progress summary.

Technology	Use and status
HTML5 / DHTML	HTML entry page plus React-rendered DOM; UI updates without a full navigation.
CSS3	Global and component styles create the Bauhaus-inspired board.
Bootstrap 5	Base CSS loaded from jsDelivr CDN; custom styles override and extend it.
JavaScript / ReactJS	Functional components, hooks, contexts, controlled forms, and async fetch calls.
NodeJS	Runtime for the backend server and asynchronous file/database operations.
ExpressJS	Middleware, authentication checks, health/auth/task routes, and JSON responses.
REST / JSON	HTTP resources: GET, POST, PUT, PATCH, DELETE; JSON bodies and results.
MongoDB / Mongoose	Primary database and schema/model layer for users and tasks.
bcryptjs / JWT	Password hashing and signed bearer tokens.
Postman	Manual API exploration using a supplied collection.
Vercel	Assumed host for the Vite frontend static build.
Render	Assumed host for the Express Node web service.
Tomcat	Can serve frontend/dist as static files; does not run Express.
DynamoDB / Cassandra	Not connected in this project; separate persistence adapters are needed.
@hello-pangea/dnd	Drag-and-drop for moving cards across board columns.

Tools and why they matter

Tool	Importance
VS Code / editor	Browse and edit separate frontend and backend code.
Node.js and npm	Install dependencies, run the API, start Vite, and build frontend assets.
Git / GitHub	Version control and source hosting. A commit-to-task webhook is not implemented.
MongoDB Atlas	Managed cloud database used by the deployed API.
Postman	Check endpoints and bearer-token behavior without using the UI.
Vercel / Render	Separate static frontend and long-running Node API deployment targets.

2. Functions and how data moves

Current functions

- Registration and login using username or email; password hashes are stored instead of plain passwords.
- Create, view, edit, move, and delete task cards; drag a card to change its status.
- Search by title and filter by high priority, overdue, assigned to me, or bug category.
- Progress counts and completion bars summarize the filtered tasks.
- Task fields include title, description, status, priority, category, assignee, due date, attachment, and timestamps.
- File attachments accept PNG, JPEG, GIF, WEBP, PDF, and TXT; images can be previewed.

Normal task request flow

Step	What happens
1	React component calls a TaskContext action.
2	TaskContext sends fetch() to VITE_BACKEND_URL with JSON and the saved bearer token.
3	Express route checks JWT and validates the requested change.
4	Mongoose runs a query on the Task model; MongoDB stores or returns the document.
5	Express returns JSON. TaskContext reloads GET /api/tasks and React redraws the board.

Registration follows a similar flow, but /api/register and /api/login are public. The API hashes a new password with bcryptjs, stores the user, and returns a JWT. The browser saves the token in localStorage.

Database structure

Collection	Important fields	Notes
users	username, email, password	Unique username/email. Password field stores bcrypt hash.
tasks	title, description, status, priority, category, attachment, assignee, dueDate	Mongoose timestamps add createdAt and updatedAt. Status/priority/category use enums.

MongoDB stores BSON documents. Mongoose models are declared in backend/models/User.js and Task.js. The connection string selects the database; in this project configuration that database is named kanban. MongoDB collections are created on first insert if absent.

3. Important files and responsibilities

Path	Purpose
SprintStack/README.md	Local setup, API routes, stack boundaries, Vercel/Tomcat notes.
backend/server.js	Express setup, CORS, JSON parsing, JWT middleware, auth routes, task REST routes, Mongo connect and local fallback.
backend/models/User.js	Mongoose User schema; unique username/email and password hash.
backend/models/Task.js	Mongoose Task schema, allowed enum values, attachment fields, timestamps, and id serialization.
backend/.env.example	Safe variable template. Actual backend/.env is secret and Git-ignored.
backend/data/local-dev.json	Local fallback database. Development only; contains user/task records and is Git-ignored.
backend/package.json / lock	Express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv; start/dev scripts and locked versions.
frontend/index.html	HTML shell, Bootstrap CDN stylesheet, and React root.
frontend/src/main.jsx	Creates React DOM root and loads the application.
frontend/src/App.jsx	Provider setup; chooses authentication screen or task board.
frontend/src/context/AuthContext.jsx	Login/register HTTP calls; stores and clears JWT and username.
frontend/src/context/TaskContext.jsx	GET/POST/PUT/PATCH/DELETE task requests and refreshes task state.
frontend/src/components/TaskBoard.jsx	Board header, filters, charts, columns, drag/drop, and modals.
frontend/src/components/Column.jsx, TaskCard.jsx	Column and draggable task-card UI.
frontend/src/components/TaskModal.jsx, TaskViewModal.jsx	Create/edit task form and task detail dialog.
frontend/src/index.css and component CSS	Global theme, board layout, responsive rules, cards, controls, dialogs.
frontend/vite.config.js	Vite dev and preview port settings (3000).
frontend/vercel.json	SPA rewrite to index.html for client-side paths.
frontend/.env.example	Documents VITE_BACKEND_URL for Vite builds.
postman/SprintStack.postman_collection.json	Health, registration, login, task CRUD/move requests and variables.

How to read the app quickly

Start at main.jsx, then App.jsx. Follow AuthProvider to AuthContext. After authentication, TaskProvider loads tasks and exposes actions to TaskBoard, TaskModal, and TaskCard. The browser never connects directly to MongoDB; only backend/server.js does.

4. Database, API, and key syntax

MongoDB connection (backend/server.js)

```
mongoose.connect(process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/sprintstack',
{ serverSelectionTimeoutMS: 2500 });
```

The backend attempts MongoDB on startup. If it cannot connect, it selects backend/data/local-dev.json for local development and reports local-file from GET /api/health. This fallback is not reliable production storage on Render; production must connect to Atlas.

Typical Mongoose query syntax

```
const tasks = await Task.find({}).sort({ createdAt: -1 });
const saved = await Task.create({ title: 'Plan sprint', status: 'To Do' });
const updated = await Task.findByIdAndUpdate(id, { status: 'Done' }, { new: true });
const removed = await Task.findByIdAndDelete(id);
```

REST API routes

Method + path	Auth	Purpose
GET /api/health	No	Health and selected storage mode.
POST /api/register	No	Create user; return token and username.
POST /api/login	No	Verify credentials; return token.
GET /api/users	Bearer	List usernames and emails.
GET /api/tasks	Bearer	List tasks.
POST /api/tasks	Bearer	Create task.
PUT /api/tasks/:id	Bearer	Edit task fields.
PATCH /api/tasks/:id/status	Bearer	Move task status.
DELETE /api/tasks/:id	Bearer	Delete task.

Key environment variables

Variable	Where	Meaning
VITE_BACKEND_URL	Vercel frontend build	Public HTTPS origin of Render API. VITE_ values are visible in client code.
MONGODB_URI	Render backend	Private Atlas URI. Never place in frontend.
JWT_SECRET	Render backend	Private signing key for JWTs.
FRONTEND_URL	Render backend	Exact Vercel frontend origin allowed by CORS.
PORT	Render runtime	Platform-provided port; app reads process.env.PORT, local fallback 5000.

5. Assumed Vercel and Render deployment

This is a deployment plan, not a claim that the services have already been deployed. Vercel and Render can deploy separate directories from the same repository. Vercel supports a root directory for a monorepo project; Render web services host Node/Express and require the app to bind to the public port.

Host/project	Configuration
Vercel frontend	Root directory: SprintStack/frontend. Framework: Vite. Install: npm install. Build: npm run build. Output: dist. Set VITE_BACKEND_URL to https://.onrender.com.
Render API	Root directory: SprintStack/backend. Build: npm install. Start: npm start. Set MONGODB_URI, JWT_SECRET, FRONTEND_URL. Use platform PORT; bind public HTTP to 0.0.0.0.
MongoDB Atlas	Provide database user credentials in MONGODB_URI. Ensure Atlas network access permits the Render service. Use production database; do not rely on JSON fallback.
CORS	Set FRONTEND_URL to the exact deployed Vercel origin, including https:// and without trailing slash.

Render readiness note

The current server uses const PORT = process.env.PORT || 5000, which is suitable for a host-provided port. Its app.listen call does not explicitly pass a host. Render documents binding the public web server to 0.0.0.0; before relying on a Render deployment, set the listen host explicitly in backend/server.js, for example:

```
app.listen(PORT, '0.0.0.0', () => {
console.log(`SprintStack API listening on ${PORT}`);
});
```

Set VITE_BACKEND_URL in Vercel before the Vite build, because Vite embeds it in the static client. Add secrets to the Render service environment, not the frontend. After changing environment variables, redeploy the affected service. Render can expose its PORT variable and Vercel monorepo projects can target separate root directories.

Local run commands

```
Backend: cd SprintStack/backend && npm install && npm start
Frontend: cd SprintStack/frontend && npm install && npm run dev
Local UI: http://localhost:3000 API: http://localhost:5000
```

Deployment references

Vercel monorepo root directories | Vite on Vercel
Render web services and port binding | Render environment variables

6. Current limits and operational notes

Area	Current behavior / limit
Task privacy	Tasks are shared among authenticated accounts; server does not store task owner or enforce per-user task permissions.
Authentication	JWT stored in localStorage, 7-day token, no email verification, password reset, refresh token, or server-side revocation.
Live updates	SprintStack re-fetches after its own actions; other open clients do not receive push updates. Original Internship_assignment uses Socket.IO.
Database fallback	Local JSON fallback is for development. Do not use it as persistent production storage on Render.
Attachments	Base64 stored in MongoDB task documents. Browser limit 5 MB; base64 expands bytes and Express JSON body limit is 8 MB. MongoDB document size also limits attachments.
Role/validation	No role system. Current APIs validate selected task fields, but registration has no email verification or password-strength policy.
Other databases	DynamoDB and Cassandra adapters are not present; MongoDB is the active database.
Tomcat	Can serve the frontend build, not the Node backend.
Testing	Frontend production build and backend syntax checks passed in the workspace. End-to-end cloud deployment has not been verified here.

Security checklist

- Keep backend/.env and backend/data/local-dev.json out of Git; the local data may include account information.
- Use a long random JWT_SECRET and a private Atlas URI on Render. Rotate credentials that have been exposed.
- Never place MONGODB_URI or JWT_SECRET in VITE_ variables. Frontend variables are public in the browser bundle.
- Limit CORS to the deployed frontend origin and scope tasks to authenticated users before real multi-user use.
- Move file attachments to object storage for production and store only metadata/URLs in MongoDB.

Git state and deployment distinction

The project is inside a Git repository at the workspace root. A Git remote enables version control and source hosting, but does not make task records change from commit messages. The webhook feature described next is future work and must be built separately.

7. Future scope GitHub commit-to-task automation

Proposed feature: when a push contains a commit message using a documented task command, GitHub sends a webhook to the Render API. The API validates the webhook, parses the message, writes a task change to MongoDB, and records the commit reference. This is not currently implemented.

Suggested event flow

Step	Future behavior
1. Configure	Add a GitHub repository webhook for push events, target Render /api/integrations/github/webhook, and assign a private webhook secret.
2. Verify	Render reads the raw request body and verifies X-Hub-Signature-256 with HMAC SHA-256 and a constant-time comparison.
3. Parse	Read each pushed commit message. Ignore messages without an allowed command. Validate title, status, priority, and task ID.
4. Apply	Create or update Task documents using Mongoose. Store commit SHA and repository to prevent duplicate processing.
5. Show	Return a quick 2xx response, then refresh clients. Add Socket.IO/SSE or short polling if open boards must update immediately.

Possible commit message convention

```
[task:create] Add password reset flow | priority=High | assignee=maya
[task:move] 66f0... task status=In Progress
[task:done] 66f0...
```

These are examples of a proposed grammar only. Before implementation, decide whether commit commands create tasks, update by Mongo task ID, close tasks, or use issue/branch references. Reject malformed commands rather than guessing.

Implementation requirements

- Add GITHUB_WEBHOOK_SECRET to Render only. Validate the signature before trusting any payload; GitHub recommends HMAC SHA-256 signature verification.
- Use the GitHub delivery ID and commit SHA as idempotency keys so webhook retries cannot create duplicate tasks.
- Limit accepted repositories and branches; validate command fields and cap message/payload sizes.
- Write an audit trail with repository, commit SHA, actor, event type, and resulting task ID.
- Choose whether webhook-created tasks are global or assigned to an authorized project/user. Current API task records have no owner.
- Add tests for valid signature, invalid signature, retry delivery, malformed message, and successful task creation.
- For immediate browser updates, use Socket.IO or Server-Sent Events, or poll the REST task list. A database write alone does not push changes to already open browsers.

GitHub security reference

GitHub documentation: validating webhook deliveries and X-Hub-Signature-256

8. Roadmap and viva summary

Priority	Future improvement	Benefit
1	GitHub webhook task commands	Turn selected commit messages into traceable task actions.
2	Task ownership and project/team roles	Prevent every authenticated account from editing the same global task list.
3	Live cross-client updates	Socket.IO or SSE can push board changes without polling.
4	Object storage for attachments	Reduce Mongo document size and improve upload/download handling.
5	Auth hardening	Email verification, password reset, strength validation, refresh/revocation strategy.
6	Testing and observability	API/UI tests, structured logs, metrics, deployment health checks.
7	Reporting and accessibility	CSV/PDF exports, workload reports, keyboard and screen-reader support.
8	Alternative database adapter	Add DynamoDB/Cassandra only if project requirements need them.

Viva quick answers

Question	Short answer
What is React used for?	Builds the interactive UI from reusable components and state.
What does Express do?	Defines middleware and REST routes in the Node API.
How does the frontend call the API?	fetch() sends JSON requests to VITE_BACKEND_URL; a bearer JWT protects task routes.
What is Mongoose?	An ODM that defines User/Task schemas and runs MongoDB queries.
What does REST mean here?	Task resources are read/changed through HTTP methods and JSON.
How does the app update after a task write?	TaskContext re-fetches tasks; React state changes and rerenders the board.
Is GitHub commit automation implemented?	No. It is proposed future scope; a signed webhook and parser still need to be built.
Why Vercel and Render?	Vercel serves the static frontend; Render runs the Node/Express API.
Why must the database be Atlas in deployment?	Render local file fallback is not durable across ephemeral instances; Atlas is the persistent store.

Short project pitch

“SprintStack is a React Kanban board backed by an Express REST API and MongoDB. JWT authentication protects task operations. Vercel serves the frontend and Render runs the API in the assumed deployment. A planned GitHub webhook can convert explicitly marked commit messages into audited task changes, with signature verification and live board refresh added as part of that future feature.”